

Cmpe 492 - Presentation 3

Mustafa Ocak - Halil İbrahim Kasapoğlu
Advisor: Atay Ozgovde

Bogazici University



Volumes

- ▶ Data inside the container is ephemeral, container state does not persist across different sessions, restarting a container leads to data loss. Same case is valid for K8s, since containers are one of most fundamental piece of the Kubernetes.
- ▶ Another problem is when multiple containers are running in a Pod and need to share files. It can be challenging to setup and access a shared filesystem across all of the containers

Volumes (Cont'd)

- ▶ At its core, a volume is a directory, possibly with some data in it, which is accessible to the containers in a pod.
- ▶ To use a volume, specify the volumes to provide for the Pod in `.spec.volumes` and declare where to mount those volumes into containers in `.spec.containers[*].volumeMounts`

Overview of Common Volume Types

- ▶ Kubernetes offers various volume types suited for different storage needs and environments.
- ▶ Choosing the correct volume type is essential for data persistence, scalability, and performance.

Volume Types

- ▶ **emptyDir:**
 - ▶ Temporary storage that exists as long as the Pod is running.
 - ▶ Useful for scratch space, caching, and ephemeral data storage.
- ▶ **hostPath:**
 - ▶ Maps a directory from the host to the Pod.
 - ▶ Suitable for specific host storage needs but not portable across nodes.
- ▶ **PersistentVolume (PV):**
 - ▶ Manages durable storage, independent of Pod lifecycle.
 - ▶ Allows data persistence across Pod restarts and rescheduling.

Cloud Storage Volume Types

- ▶ **awsElasticBlockStore:**

- ▶ Integrates with AWS Elastic Block Storage (EBS) to provide persistent storage.
- ▶ Useful for stateful applications requiring durable, block-level storage on AWS.

- ▶ **gcePersistentDisk:**

- ▶ Google Cloud Engine's Persistent Disk service.
- ▶ Offers high-performance storage that can be easily mounted to multiple Pods.

- ▶ **azureDisk:**

- ▶ Azure's managed disk service for durable, high-performance storage.
- ▶ Provides single-Pod access on Azure.

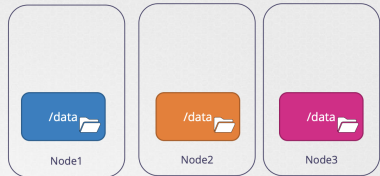
Network-Based Volume Types

- ▶ **NFS** (Network File System):
 - ▶ Allows storage over a network, accessible by multiple Pods on different nodes.
 - ▶ Suitable for scenarios requiring shared storage accessible across the cluster.
- ▶ **CIFS/SMB** (Common Internet File System):
 - ▶ Enables mounting SMB file shares, often used with Windows systems.
 - ▶ Suitable for integration with Microsoft storage services.

Volume Types

| Volume Types

```
volumes:  
- name: data-volume  
  hostPath:  
    path: /data  
    type: Directory
```



SCALEIO



Persistent Volume Requirements

- ▶ Storage should not depend on the pod lifecycle. Data should persist even if the pod restarts or terminates.
- ▶ Storage must be available on all nodes, providing shared storage. Should allow data sharing among multiple pods.
- ▶ Storage needs to survive even if the cluster crashes. Ideal for databases, logs, and other stateful services in Kubernetes.

What is Persistent Volumes (PV)?

- ▶ A Persistent Volume (PV) is a storage resource in Kubernetes that is abstracted from the lifecycle of pods and can retain data beyond the lifespan of individual containers.
- ▶ Provides a solution for long-term storage, enabling data persistence across pods and workloads.

Types of Persistent Volumes Provisioning

- ▶ **Static Provisioning:** Pre-defined storage created by administrators. PVs are created manually before they are claimed.
 - ▶ + Allows full control over storage configuration.
 - ▶ - Manual process, less flexible, requires administrator intervention.
- ▶ **Dynamic Provisioning:** PVs are created automatically based on storage classes, triggered by Persistent Volume Claims (PVCs).
 - ▶ + Automated, scales easily, on-demand provisioning based on storage classes.
 - ▶ - Dependent on storage class availability, less granular control.

PV Deployment Methods in Kubernetes

- ▶ **Local Storage:** PVs are backed by local disks; suitable for single-node applications with low latency needs.
- ▶ **NFS (Network File System):** Network-attached, shared storage for multi-node accessibility.
- ▶ **Cloud Storage Solutions:** Integrates with cloud provider-managed storage (AWS EBS, Google Cloud Persistent Disks, Azure Disks).
- ▶ **Third-Party Storage Systems:** Solutions like Ceph, GlusterFS, and Portworx that provide high-availability storage.

Persistent Volume Claims (PVC)

- ▶ Persistent Volume Claims (PVC) are requests for storage by users.
- ▶ A PVC specifies storage requirements such as access mode and storage capacity.
- ▶ Once a PVC is created, Kubernetes binds it to an available Persistent Volume (PV) based on matching criteria.

Example: Pod with hostPath Volume

- ▶ A basic Pod definition that uses a hostPath volume:

```
apiVersion: v1
kind: Pod
metadata:
  name: sample-pod
spec:
  containers:
    - name: busybox
      image: busybox
      command: ["/bin/sh", "-c", "echo Hello
        Kubernetes! > /data/hello.txt"]
      volumeMounts:
        - name: my-volume
          mountPath: /data
  volumes:
    - name: my-volume
      hostPath:
        path: /tmp/data
        type: Directory
```

Persistent Volume Configuration

- Defining a Persistent Volume (PV) with specific storage requirements:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /mnt/data
    type: Directory
```

Persistent Volume Claim (PVC) Example

- Defining a PVC that requests storage from an existing PV:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

Binding Process

- ▶ When a PVC is created, Kubernetes finds a suitable PV based on access modes, storage capacity, and other parameters.
- ▶ Once a PV is bound to a PVC, it cannot be bound to another PVC.
- ▶ Binding is essential to ensure data availability and accessibility.

Persistent Volume Reclaim Policies

- ▶ PVs have Reclaim Policies that define how the storage is treated when a PVC is deleted.
 - ▶ **Retain**: Keeps the data; manual cleanup is needed.
 - ▶ **Delete**: Removes the underlying storage.
 - ▶ **Recycle**: Clears the storage and makes it available for new claims.
- ▶ Select the appropriate reclaim policy based on data retention needs.

Storage Class in Kubernetes

- ▶ A Storage Class provides a way to describe the "classes" of storage available in Kubernetes.
- ▶ It defines parameters like provisioner, type, and reclaim policy for dynamically created volumes.
- ▶ It allows administrators to configure different storage backends (e.g., block, file, object storage) with different performance and access features.

Storage Class Example

- ▶ A Storage Class that defines the provisioner and the volume's type:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
provisioner: kubernetes.io/gce-pd
reclaimPolicy: Retain
volumeBindingMode: Immediate
parameters:
  type: pd-standard
```


How Storage Class Works with PVC

- ▶ When a PersistentVolumeClaim (PVC) is created, Kubernetes matches it with an available PersistentVolume that meets the storage requirements and matches the Storage Class.
- ▶ If no matching PV exists, and the PVC specifies a 'storageClassName', Kubernetes will create a new PV dynamically using the Storage Class.

Working with Persistent Volume Claims

- ▶ Commands for managing PVCs:
 - ▶ `kubectl get persistentvolumeclaim`
 - ▶ `kubectl delete persistentvolumeclaim <name>`
- ▶ Example:

```
kubectl get pvc  
kubectl delete pvc myclaim
```

Secure Kubernetes

- ▶ Who can access to kube-apiserver(authN)?
- ▶ What can they do with kube-apiserver(authZ)?

Account Types

In Kubernetes, a user represents an identity that can interact with the cluster. However, unlike traditional systems, Kubernetes does not have a built-in way to "register" users (like adding user accounts). Instead, users are usually represented by client certificates or authentication tokens.

- ▶ User Accounts (External Identities)
- ▶ Service Accounts (Internal Identities)

Account Types

- ▶ Kubernetes doesn't have an internal system for managing user accounts directly. Instead, you authenticate as a "user" via kubectl using a client certificate or token.
- ▶ When you use kubectl, you are actually acting as a user that is identified by the credentials (certificate or token) you are using to access the cluster.
- ▶ Kubernetes has Service Accounts, which are identities for applications or system components within the cluster.
- ▶ Service accounts are often used to assign permissions to pods, controllers, or other Kubernetes components (e.g., kubelet, kube-proxy).
- ▶ These are managed within the cluster and can be assigned roles using RoleBindings or ClusterRoleBindings.



Symmetric Encryption

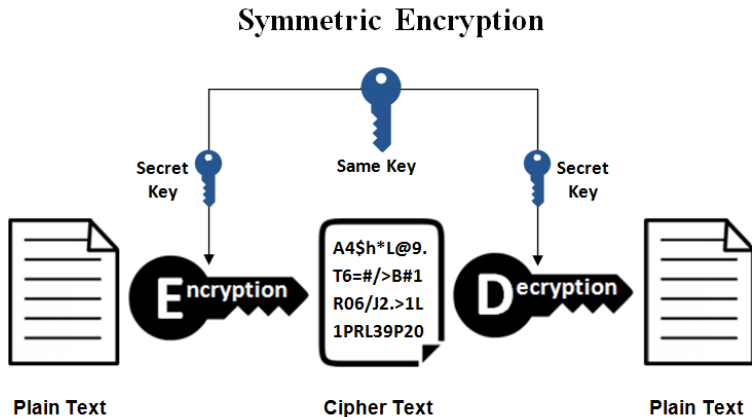


Figure 1: Symmetric Encryption

How Asymmetric Encryption Works

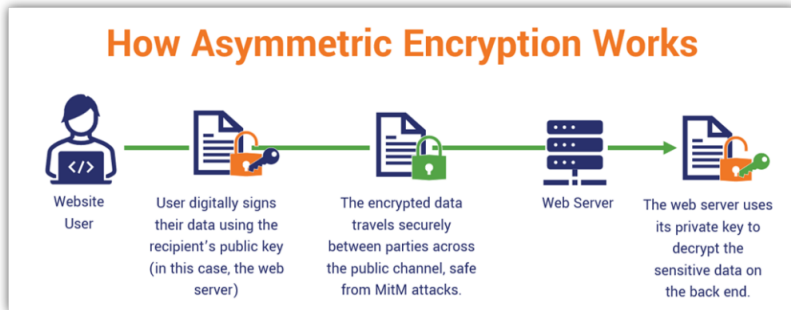


Figure 2:

Asymmetric Encryption - SSH

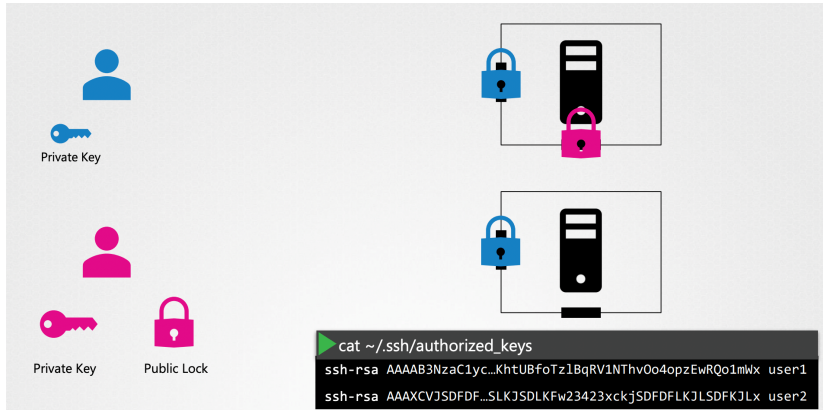
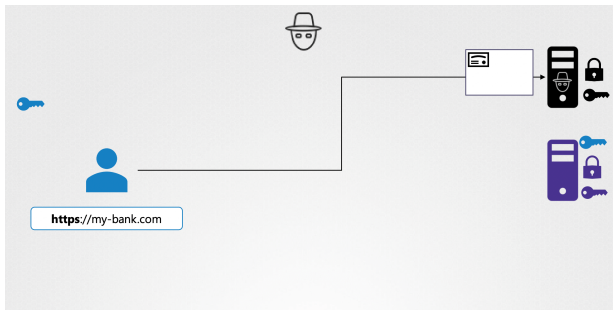


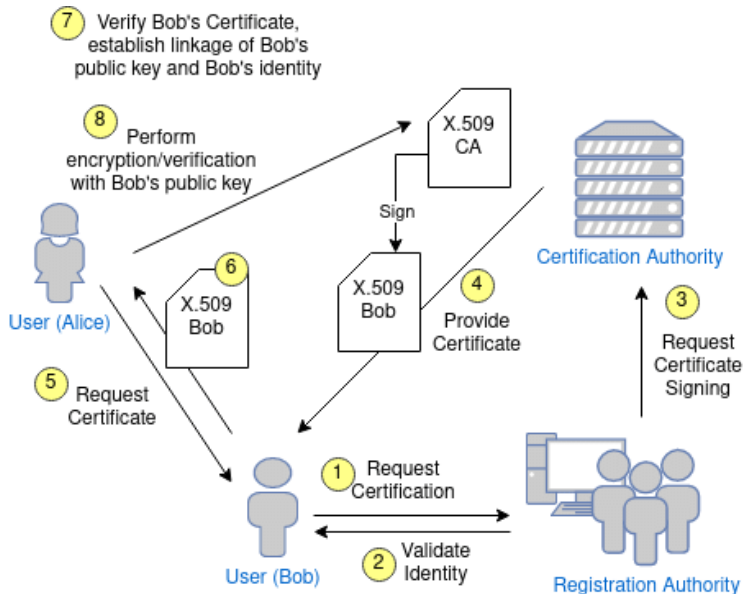
Figure 3: Asymmetric Encryption

Certificates

A TLS certificate is a type of digital certificate issued by a certificate authority (CA) to certify the ownership of a public key. TLS certificates usually contain the following information: The subject domain name (for example, my-bank.com) The name of the issuing CA.

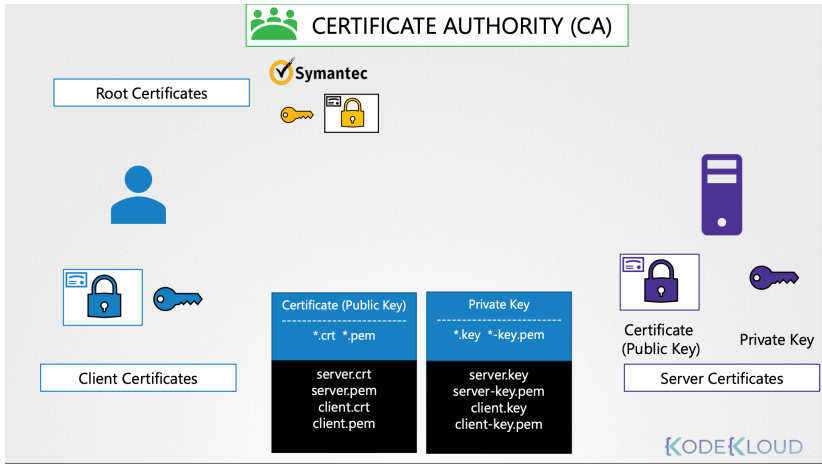


Public Key Infrastructure

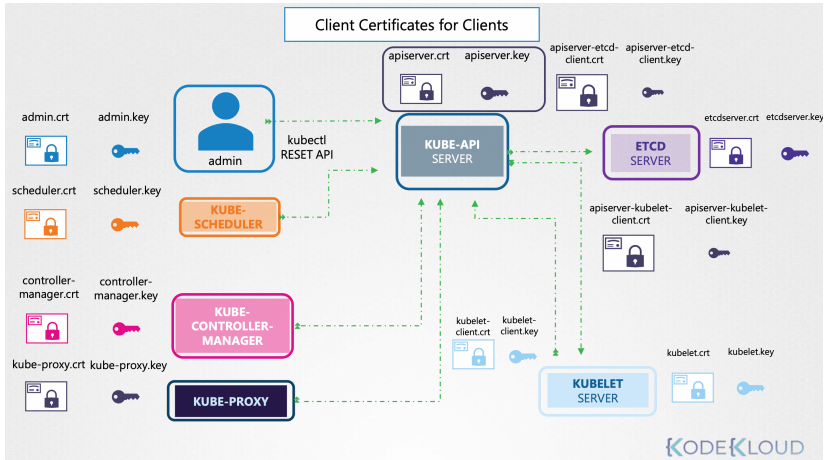


General Scheme in K8s

By this way, we won't need a static username, password or tokens.



General Scheme in K8s



General Scheme in K8s

Each Kubernetes component (e.g., admin, kube-scheduler, kube-controller-manager, kube-proxy) uses a client certificate and private key to authenticate itself when communicating with the API server.

Generate Cert. For CA



CERTIFICATE AUTHORITY (CA)

Generate Keys



ca.key

```
openssl genrsa -out ca.key 2048  
ca.key
```

Certificate
Signing
Request



ca.csr

```
openssl req -new -key ca.key -subj "/CN=KUBERNETES-CA" -out ca.csr  
ca.csr
```

Sign
Certificates



ca.crt

```
openssl x509 -req -in ca.csr -signkey ca.key -out ca.crt  
ca.crt
```

Generate Cert. For Admin

 ca.key  ca.crt

ADMIN USER

 admin.key

Generate Keys

```
openssl genrsa -out admin.key 2048
```

admin.key

Certificate Signing Request

 admin.csr

```
openssl req -new -key admin.key -subj \"/CN=kube-admin" -out admin.csr
```

admin.csr

Sign Certificates

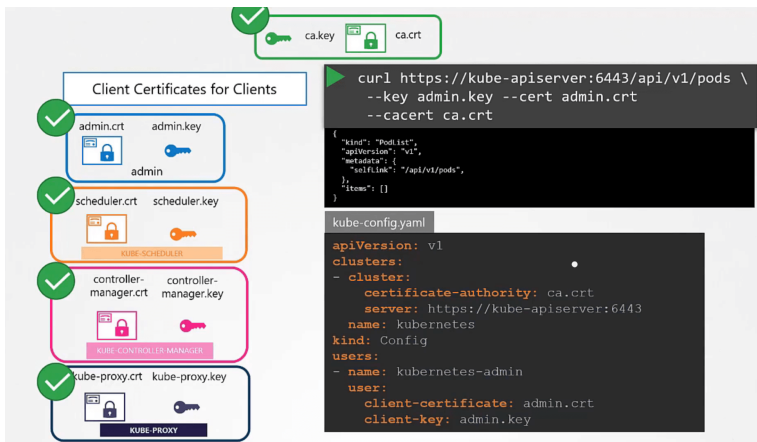
 admin.crt

```
openssl x509 -req -in admin.csr -CA ca.crt -CAkey ca.key -out admin.crt
```

admin.crt

Sending Request using Cert.

- ▶ Auth parameters
- ▶ Using KubeConfig



Kube-apiserver Configuration

KUBE API SERVER

apiserver.crt apiserver.key



KUBE-API SERVER

apiserver-kubelet-
client.crt apiserver-kubelet-
client.key



apiserver-etcd-
client.crt apiserver-etcd-
client.key



KUBE-API SERVER

```
ExecStart=/usr/local/bin/kube-apiserver \\  
--advertise-address=${INTERNAL_IP} \\  
--allow-privileged=true \\  
--apiserver-count=3 \\  
--authorization-mode=Node,RBAC \\  
--bind-address=0.0.0.0 \\  
--enable-swagger-ui=true \\  
--etcd-cafile=/var/lib/kubernetes/ca.pem \\  
--etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \\  
--etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \\  
--etcd-servers=https://127.0.0.1:2379 \\  
--event-ttl=1h \\  
--kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\  
--kubelet-client-certificate=/var/lib/kubernetes/apiserver-etcd-client.crt \\  
--kubelet-client-key=/var/lib/kubernetes/apiserver-etcd-client.key \\  
--kubelet-https=true \\  
--runtime-config=api/all \\  
--service-account-key-file=/var/lib/kubernetes/service-account.pem \\  
--service-cluster-ip-range=10.32.0.0/24 \\  
--service-node-port-range=30000-32767 \\  
--client-ca-file=/var/lib/kubernetes/ca.pem \\  
--tls-cert-file=/var/lib/kubernetes/apiserver.crt \\  
--tls-private-key-file=/var/lib/kubernetes/apiserver.key \\  
--v=2
```



The Hard Way vs Manifest File

"The Hard Way"

```
cat /etc/systemd/system/kube-apiserver.service
```

```
[Service]
ExecStart=/usr/local/bin/kube-apiserver \
--advertise-address=172.17.0.32 \
--allow-privileged=true \
--apiserver-count=3 \
--authorization-mode=Node,RBAC \
--bind-address=0.0.0.0 \
--client-ca-file=/var/lib/kubernetes/ca.pem \
--enable-swagger-ui=true \
--etcd-cafile=/var/lib/kubernetes/ca.pem \
--etcd-certfile=/var/lib/kubernetes/kubernetes.pem \
--etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \
--event-ttl=1h \
--kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \
--kubelet-client-key=/var/lib/kubernetes/kubernetes-key.pem \
--kubelet-https=true \
--service-node-port-range=30000-32767 \
--tls-cert-file=/var/lib/kubernetes/kubernetes.pem \
--tls-private-key-file=/var/lib/kubernetes/kubernetes-key.pem
-v=2
```

kubeadm

```
cat /etc/kubernetes/manifests/kube-apiserver.yaml
```

```
spec:
  containers:
  - command:
    - kube-apiserver
    - --authorization-mode=Node,RBAC
    - --advertise-address=172.17.0.32
    - --allow-privileged=true
    - --client-ca-file=/etc/kubernetes/pki/ca.crt
    - --disable-admission-plugins=PersistentVolumeLabel
    - --enable-admission-plugins=NodeRestriction
    - --enable-bootstrap-token-auth=true
    - --etcd-cafile=/etc/kubernetes/pki/etcd/ca.crt
    - --etcd-certfile=/etc/kubernetes/pki/apiserver-etcd-client.c
    - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.ke
    - --etcd-servers=https://127.0.0.1:2379
    - --insecure-port=0
    - --kubelet-client-certificate=/etc/kubernetes/pki/apiserver-
    - --kubelet-client-key=/etc/kubernetes/pki/apiserver-kubelet-
    - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hos
    - --proxy-client-cert-file=/etc/kubernetes/pki/front-proxy-cl
    - --proxy-client-key-file=/etc/kubernetes/pki/front-proxy-cl
    - --requestheader-allowed-names=front-proxy-client
```

Table

Default CN	Parent CA	O (in Subject)	kind	hosts (SAN)
kube-etcd	etcd-ca		server, client ["][etcdbug]	localhost, 127.0.0.1
kube-etcd-peer	etcd-ca		server, client	<hostname>, <Host_IP>, localhost, 127.0.0.1
kube-etcd-healthcheck-client	etcd-ca		client	
kube-apiserver-etcd-client	etcd-ca	system:masters	client	
kube-apiserver	kubernetes-ca		server	<hostname>, <Host_IP>, <advertise_IP>, [1]
kube-apiserver-kubelet-client	kubernetes-ca	system:masters	client	
front-proxy-client	kubernetes-front-proxy-ca		client	

Default CN	recommend key path	recommended cert path	command	key argument	cert argument
etcd-ca		etcd/ca.crt	kube-apiserver		-etcd-cafile
etcd-client	apiserver-etcd-client.key	apiserver-etcd-client.crt	kube-apiserver	-etcd-keyfile	-etcd-certfile
kubernetes-ca		ca.crt	kube-apiserver		-client-ca-file
kube-apiserver	apiserver.key	apiserver.crt	kube-apiserver	-tls-private-key-file	-tls-cert-file
apiserver-kubelet-client		apiserver-kubelet-client.crt	kube-apiserver		-kubelet-client-certificate
front-proxy-ca		front-proxy-ca.crt	kube-apiserver		-requestheader-client-ca-file
front-proxy-client	front-proxy-client.key	front-proxy-client.crt	kube-apiserver	-proxy-client-key-file	-proxy-client-cert-file
etcd-ca		etcd/ca.crt	etcd		-trusted-ca-file, -peer-trusted-ca-file
kube-etcd	etcd/server.key	etcd/server.crt	etcd	-key-file	-cert-file
kube-etcd-peer	etcd/peer.key	etcd/peer.crt	etcd	-peer-key-file	-peer-cert-file
etcd-ca		etcd/ca.crt	etcdctl[2]		-cacert
kube-etcd-healthcheck-client	etcd/healthcheck-client.key	etcd/healthcheck-client.crt	etcdctl[2]	-key	-cert

Certificate API workflow

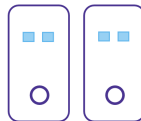
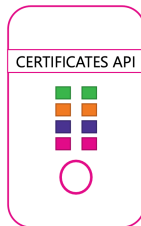
Instead of manually signing certificates signing requests, kubernetes provide Certificates API, to make the process easier.

1. Create CertificateSigningRequest Object

2. Review Requests

3. Approve Requests

4. Share Certs to Users



Certificate API workflow

jane.csr

```
-----BEGIN CERTIFICATE REQUEST-----
MIICMDCAUACAQAwEzERMA8GA1UEAwwIbWV3LXVzZXIwggEiMA8GCSqGSIb3DQEBAQUAA4IBDwAwggEKAoIBAQQD00WJh+DXsaJSIrjpn05vRIBp1nzg+6xc9+UvWkKi0
LFC27t+1eEnONSMuq99NevmMEOnrDUO/thyVqP2w2XNIDRXjYyF40FbmD+SzmYCK
9w0BAQsFAAOCAQEAS91S6C1uxTuf5BBSYU7QFQHuzalNxAdYsa0RRQNhHwHq614
hOK4a2zyNy144001jyaD6tUM8DSxkr8BLK8K3srRetJq15rLZy9LRVrsJghD4gY
P9NL+aDRSxROVSqBab2nWeyPMGc1STFS31+NSNMLQ2+RMnjDQ17juPE1c8/dhk
Wz2EUM6UwzykrhThmTVza1MY0R0+DNEvYVie+0W9/VE1t+FSGjHSLSVUvIIdqiy
413E/y3qL7iHAcuH30sVpUuH0tS4Gqs8qCcbE56CC5DhPGZIpUbnKUpAwka+8E
vuQ07j6+hpKxmUFAeXgUwcdAla77ju/TD1Cw==
-----END CERTIFICATE REQUEST-----
```

cat jane.csr | base64

```
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB5RVFVRVNULS0
tLS0KTU1JQ1dEQ0NBVUFDQVFBd0V6RVJNQThHQTFVRU
F3d01ibVYzTFhwe1pY5XdnZ0VpTUEwR0NTCudTSWIZR
FFFQgpbUVVBQTRJQkR3QXdnZ0VlQW9JQkFRRE8wV0pX
K0RYc0FKU01yanB0bzV2Uk1CcGxuemcrNnhj0StVVnd
rS2kwCkxmQzI3dCsxZUVuT041TXVxOT10ZXZtTUVpbN
J
```

jane-csr.yaml

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: jane
spec:
  expirationSeconds: 600 #seconds
  usages:
    - digital signature
    - key encipherment
    - server auth
  request:
```

```
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB5RVFVRVNULS0
FVRVNULS0tLS0KTU1JQ1dEQ0NBVUFDQVFBd0V6RVJNQTh
d0V6RVJNQThHQTFVRUF3d01ibVYzTFhwe1pY5XdnZ0Vp
TUEwR0NTCudTSWIZRFFFQgpbUVVBQTRJQkR3QXdnZ0Vl
QW9JQkFRRE8wV0pXK0RYc0FKU01yanB0bzV2Uk1CcGxue
mcrNnhj0StVVndrS2kwCkxmQzI3dCsxZUVuT041TXVxOT
10ZXZtTUVpbNj
```

Certificate API workflow

```
▶ kubectl get csr
```

NAME	AGE	REQUESTOR	CONDITION
jane	10m	admin@example.com	Pending

```
▶ kubectl certificate approve jane
```

```
jane approved!
```

Certificate API workflow

```
► kubectl get csr jane -o yaml
```

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  creationTimestamp: 2019-02-13T16:36:43Z
  name: new-user
spec:
  groups:
  - system:masters
  - system:authenticated
  expirationSeconds: 600
  usages:
  - digital signature
  - key encipherment
  - server auth
  username: kubernetes-admin
status:
  certificate:
LS0tLS1CRUdJTiBDRVJUSUZ7Q0FURS0tLS0tCk1JSURDakNDQWZLZ0F3SUJBZ01VRmwy
Q2wxYXoxaw1SM3JNVIsreFRYUowU3dnd0RRWUpLb1pJaHZjTkFRRUwKQ1FBd0ZURVRN
QkVHQTFVRUF4TUtHMTZpLWlkVpYUmxjekFIRncweE9UQXlNVE14TmpleU1EQmFGd1dn
Y0ZFeD12ajNuSXZ3eFd0S1NIRm5sU041c0t5Z0VxUkwzTFM5V29Ge1hHZDdhCm1EZ2F0
MVMVRFBXTVhJN09FVnVjSWc1Yk4weEVHTkVwRU5tdU1BN1ZweHVJS1h6aG9ldDY0MEd1
MGU0YXFKWlKwVmBjBvRTRFCY3dod2xic0I1ND0KLS0tLS1FTkQgQ0VSVElGSUNBEUT
LS0tLQo=
  conditions:
  - lastUpdateTime: 2019-02-13T16:37:21Z
    message: This CSR was approved by kubectl certificate approve.
    reason: KubectlApprove
```

```
► echo "LS0...Qo=" | base64 --decode
```

```
-----BEGIN CERTIFICATE-----
MIICWDCCAUAQAQAwEzERMA8GA1UEAwIbmV3LXVzZXIwgg
AQUAA4IBDwAwggEKAoIBAQQD00WJW+DXsAJSIrjpNo5vRIB
Lfc27t+1eEnON5Muq99NevmMEOnrDUO/thyVqP2w2XNIDR
y3BiHhB93MJ70ql3UTvZ8TELqyaDknR1/jv/SxgXkok0AB
IF5nxAttMVkDPQ7NbeZRG43b+QwLVGR/z6DWO0fJnbfezOt
EcXAwqChjBLkz2BHPR4J89D6Xb8k39pu6jpyngV6uP0tI
j2qEL+hZEwkkFz80lNNtyT5LxMqENDCnIgwC4GZiRgbrAg
9w0BAQsFAAOCAQEA5iS6C1uxTuf5BBYSU7QFQHUza1Nxh
hOK4a2zyNy14400ijyaD6tUW8DSxkr8BLK8Kg3srREtJq1
P9NL+aDRSxROVSqBaB2nweYpM5cJ5TF53lesNSNMLQ2++R
Wr2EUM6UawzykrdHImwTv2m1MY0R+DnTV1Yie+0H9/YE1t
4l3E/y3qL7lWfAcuH30sVpUUnQISMDQs0QWCSbE56CCSDh
vwQ07jG+hpknxmuFAeXxglUwodAlaJ7ju/TDIcw==
-----END CERTIFICATE-----
```


Who Is Responsible ?



```
cat /etc/kubernetes/manifests/kube-controller-manager.yaml

spec:
  containers:
  - command:
    - kube-controller-manager
    - --address=127.0.0.1
    - --cluster-signing-cert-file=/etc/kubernetes/pki/ca.crt
    - --cluster-signing-key-file=/etc/kubernetes/pki/ca.key
    - --controllers=*,bootstrapsigner,tokencleaner
    - --kubeconfig=/etc/kubernetes/controller-manager.conf
    - --leader-elect=true
    - --root-ca-file=/etc/kubernetes/pki/ca.crt
    - --service-account-private-key-file=/etc/kubernetes/pki/sa.key
    - --use-service-account-credentials=true
```

Kubeconfig

Kubernetes uses a YAML file called kubeconfig to store cluster authentication information for kubectl . kubeconfig contains a list of contexts to which kubectl refers when running commands. By default, the file is saved at \$HOME/.kube/config.

KubeConfig File

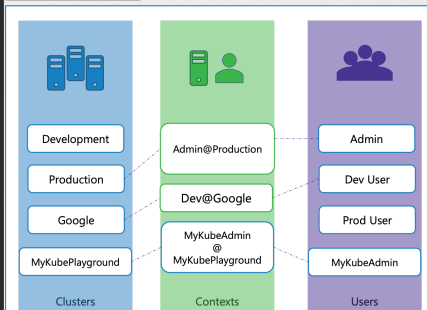
```
apiVersion: v1
kind: Config

clusters:
- name: my-kube-playground
  cluster:
    certificate-authority: ca.crt
    server: https://my-kube-playground:6443

contexts:
- name: my-kube-admin@my-kube-playground
  context:
    cluster:
    user:

users:
- name: my-kube-admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```

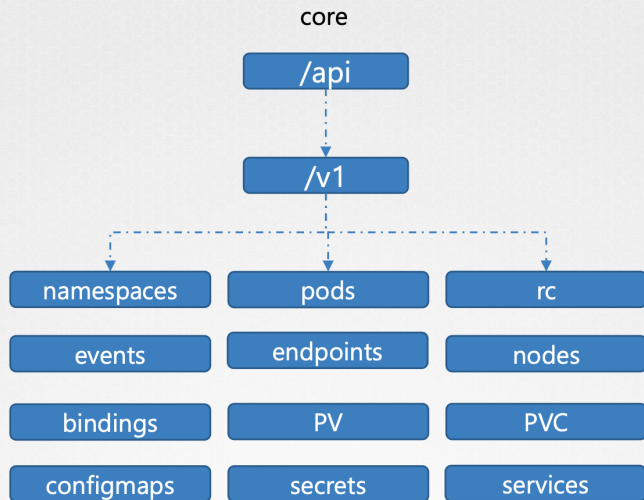
\$HOME/.kube/config



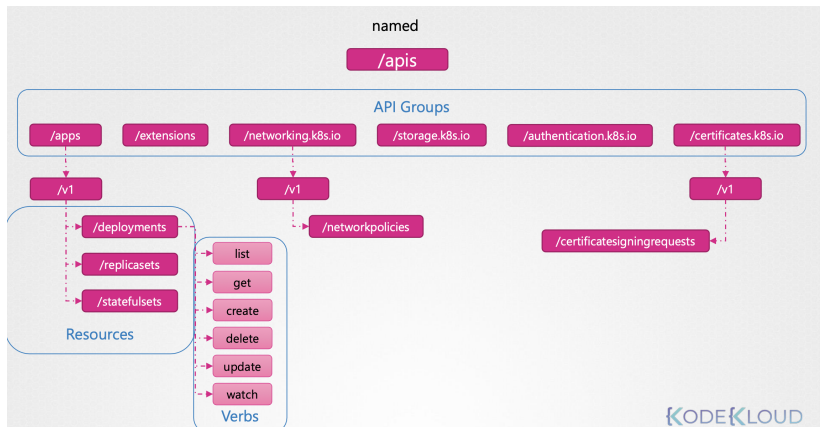
API Groups

The core group contains essential resources that are integral to the platform's operation. Its API endpoint is located at `/api/v1`. On the other hand, the named groups allow you to extend Kubernetes functionality by introducing custom resources and controllers. Its API endpoint is `/apis/$GROUP_NAME/$VERSION`.

API Groups - Core



API Groups - Named



Authorization

- ▶ Node Level
- ▶ ABAC
- ▶ RBAC ★
- ▶ Webhook



Admins

```
▶ kubectl get pods
```

NAME	STATUS	ROLES	AGE	VERSION
worker-1	Ready	<none>	5d21h	v1.13.0
worker-2	Ready	<none>	5d21h	v1.13.0

```
▶ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
worker-1	Ready	<none>	5d21h	v1.13.0
worker-2	Ready	<none>	5d21h	v1.13.0

```
▶ kubectl delete node worker-2
```

Node worker-2 Deleted!



Developers

```
▶ kubectl get pods
```

NAME	STATUS	ROLES	AGE	VERSION
worker-1	Ready	<none>	5d21h	v1.13.0
worker-2	Ready	<none>	5d21h	v1.13.0

```
▶ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
worker-1	Ready	<none>	5d21h	v1.13.0
worker-2	Ready	<none>	5d21h	v1.13.0

```
▶ kubectl delete node worker-2
```

Error from server (Forbidden): nodes "worker-1" is forbidden: User "developer" cannot delete resource "nodes"



Bots

```
▶ kubectl get pods
```

Error from server (Forbidden): nodes "worker-1" is forbidden: User "Bot-1" delete resource "nodes"

```
▶ kubectl get nodes
```

Error from server (Forbidden): nodes "worker-1" is forbidden: User "Bot-1" delete resource "nodes"

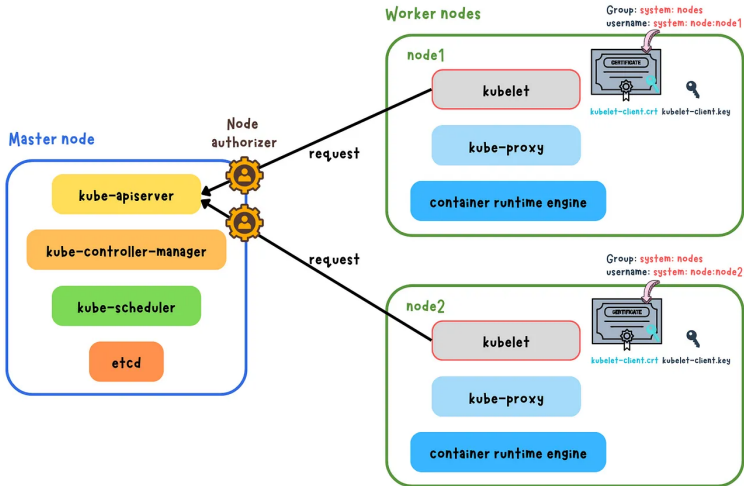
```
▶ kubectl delete node worker
```

Error from server (Forbidden): nodes "worker-1" is forbidden: User "Bot-1" delete resource "nodes"

Node Authorization

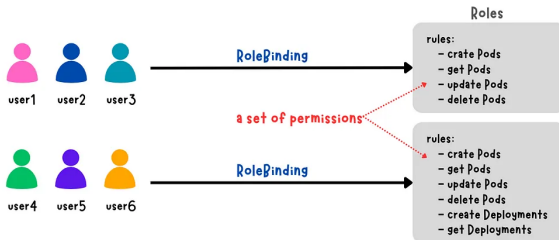
Node Authorization

a specific type of authorization mode that is used to **authorize API requests made by kubelets**



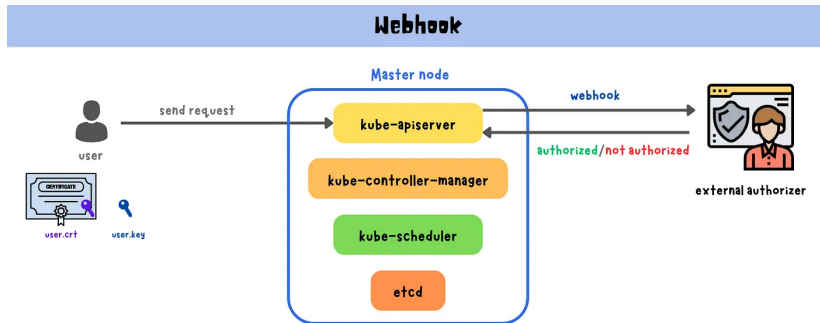
RBAC (Role-Based Access Control)

create roles and bind them to the appropriate entities



Webhook

Webhook authorization mode allows for custom authorization logic by delegating the authorization decision to an external HTTP service, known as a webhook.



Always Mode

- ▶ AlwaysAllow: mode allows all requests without any further authorization checks. This mode can be useful in certain development or testing environments, where ease of access is prioritized over security. However, it is not recommended for use in production environments.
- ▶ AlwaysDeny: mode denies all requests without any further authorization checks. This mode can be useful in highly secure environments where strict access control is critical, or in situations where authorization is not required at all.

RBAC

IRBAC

dev-user → Developer

Namespace: default

- ✓ Can view PODs
- ✓ Can create PODs
- ✓ Can Delete PODs
- ✓ Can Create ConfigMaps

```
developer-role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "get", "create", "update", "delete"]
- apiGroups: [""]
  resources: ["ConfigMap"]
  verbs: ["create"]

devuser-developer-binding.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: devuser-developer-binding
subjects:
- kind: User
  name: dev-user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer
  apiGroup: rbac.authorization.k8s.io
```

```
➤ kubectl create -f devuser-developer-binding.yaml
```

```
➤ kubectl get roles
```

NAME	AGE
developer	4s

```
➤ kubectl get rolebindings
```

NAME	AGE
devuser-developer-binding	24s

```
➤ kubectl describe role developer
```

Name: developer			
Labels: <none>			
Annotations: <none>			
PolicyRule:			
Resources	Non-Resource URLs	Resource Names	Verbs
-----	-----	-----	-----
ConfigMap	[]	[]	[create]
Pods	[]	[]	[get watch list create delete]

RBAC

```
➤ kubectl auth can-i create deployments
```

```
yes
```

```
➤ kubectl auth can-i delete nodes
```

```
no
```

```
➤ kubectl auth can-i create deployments --as dev-user
```

```
no
```

```
➤ kubectl auth can-i create pods --as dev-user
```

```
yes
```

```
➤ kubectl auth can-i create pods --as dev-user --namespace test
```

```
no
```

Resource Names



blue



green



orange



purple



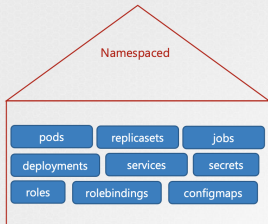
pink

```
developer-role.yaml
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "create", "update"]
  resourceNames: ["blue", "orange"]
```

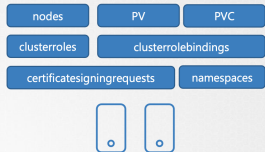
Cluster Roles

Namespace



```
▶ kubectl api-resources --namespaced=true
```

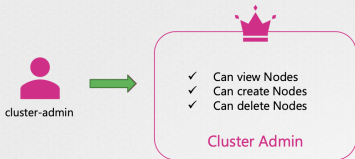
Cluster Scoped



```
▶ kubectl api-resources --namespaced=false
```

Cluster Roles

clusterrolebinding



```
kubectl create -f cluster-admin-role-binding.yaml
```

cluster-admin-role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-administrator
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["list", "get", "create", "delete"]
```

cluster-admin-role-binding.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-admin-role-binding
subjects:
- kind: User
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-administrator
  apiGroup: rbac.authorization.k8s.io
```

Kubernetes RBAC: Roles vs ClusterRoles

Scope:

- ▶ **Roles:** They are namespace-specific. This means that when you define a `Role`, it can only be used to grant access to resources within a specific namespace. If you have multiple namespaces and need to grant similar permissions in each, you'd have to define a `Role` in each of those namespaces.
- ▶ **ClusterRoles:** As the name implies, `ClusterRoles` have a cluster-wide scope. When you define a `ClusterRole`, it can be used to grant access to resources across all namespaces in the cluster. It's useful for tasks that need a broader scope, like cluster administration.

Kubernetes RBAC: Roles vs ClusterRoles

Use Cases:

- ▶ **Roles:** Ideal for specific namespace tasks. For example, if you have a team dedicated to managing a particular application within its namespace, you'd use a `Role` to grant them necessary permissions within that namespace.
- ▶ **ClusterRoles:** Suited for cluster-level tasks or multi-namespace tasks. For instance, granting read access to all `Pods` in the cluster or giving administrative privileges.

Kubernetes RBAC: Roles vs ClusterRoles

Binding:

- ▶ **Roles:** They bind to subjects (users, groups, or service accounts) using `RoleBindings` within the same namespace.
- ▶ **ClusterRoles:** They can be bound to subjects using `ClusterRoleBindings`, granting permissions cluster-wide. Interestingly, `ClusterRoles` can also be bound to subjects within a specific namespace using `RoleBindings`, enabling reuse of common permissions patterns.

Container Security - Private Registry

image: `docker.io/nginx/nginx`



Registry User/
Account Image/
Repository

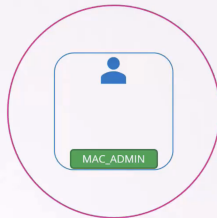
nginx-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: private-registry.io/apps/internal-app
      imagePullSecrets:
        - name: regcred
```

```
kubectl create secret docker-registry regcred \
  --docker-server= private-registry.io \
  --docker-username= registry-user \
  --docker-password=registry-password \
  --docker-email= registry-user@org.com
```

Security Context

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep", "3600"]
      securityContext:
        runAsUser: 1000
        capabilities:
          add: ["MAC_ADMIN"]
```



Note: Capabilities are only supported at the container level and not at the POD level

Container Security

Container Security:

- ▶ Focuses on protecting containerized applications and their environments from security threats.
- ▶ Involves best practices like image scanning, least privilege, and network segmentation to prevent vulnerabilities.
- ▶ Regularly updated images and monitoring help identify and mitigate risks.

Security Context:

- ▶ Defines security settings for a container or pod in Kubernetes.
- ▶ Configures permissions like running as a non-root user, setting file system privileges, and restricting capabilities.
- ▶ Example settings include 'runAsUser', 'runAsGroup', and 'readOnlyRootFilesystem'.



gVisor for Container Isolation Security

Purpose: gVisor provides a container runtime that enforces strict isolation, creating a security boundary between the host and container.

How gVisor Enhances Security:

- ▶ Kernel Isolation: Implements a user-space kernel to intercept and manage system calls, preventing containers from accessing the host kernel directly.
- ▶ Reduced Attack Surface: By isolating system calls, gVisor minimizes exposure to kernel vulnerabilities.
- ▶ Protection Against Container Escapes: Acts as a security barrier, making it more difficult for attackers to exploit containers to access the host.

gVisor for Container Isolation Security

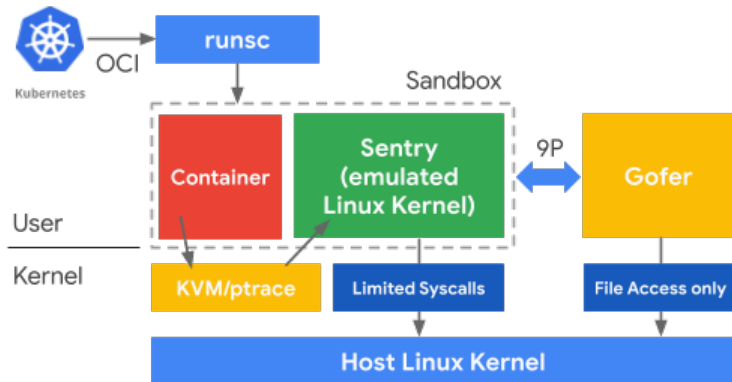


Figure 4: gVisor diagram

Traditional Container Isolation

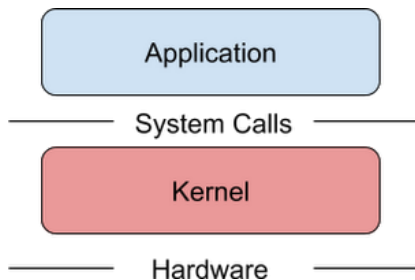


Figure 5: Traditional Container Isolation

Applications that run in traditional Linux containers access system resources in the same way that regular (non-containerized) applications do: by making system calls directly to the host kernel. The kernel runs in a privileged mode that allows it to interact with the necessary hardware and return results to the application.

VM Container Isolation

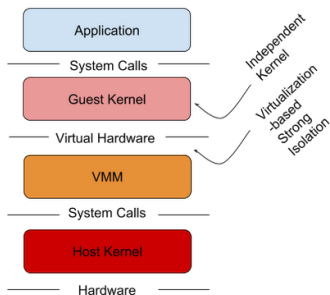


Figure 6: VM Container Isolation

One approach to improve container isolation is to run each container in its own virtual machine (VM). This gives each container its own "machine," including kernel and virtualized devices, completely separate from the host. Even if there is a vulnerability in the guest, the hypervisor still isolates the host, as well as other applications/containers running on the host.

gVisor Container Isolation

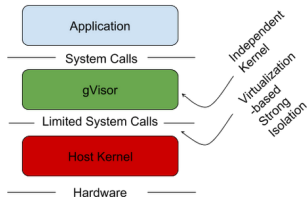


Figure 7: gVisor Container Isolation

gVisor is more lightweight than a VM while maintaining a similar level of isolation. The core of gVisor is a kernel that runs as a normal, unprivileged process that supports most Linux system calls. This kernel is written in Go, which was chosen for its memory- and type-safety. Just like within a VM, an application running in a gVisor sandbox gets its own kernel and set of virtualized devices, distinct from the host and other sandboxes.